

Phân tích thuật toán MLEUP

Ý tưởng, giới hạn và mức phù hợp với dữ liệu Trivago

Bản tổng hợp từ bài báo MLEUP và dữ liệu Trivago hiện có

24 tháng 9 năm 2026

1 Bài toán và ý tưởng chính

MLEUP giải bài toán dự đoán item tiếp theo từ một phiên ẩn danh. Đầu vào nguyên bản là chuỗi item có thứ tự trong phiên, ký hiệu $S = [v_1, \dots, v_l]$. Mô hình bổ sung thông tin từ các phiên lịch sử liên quan và thuộc tính item để xử lý tình trạng phiên ngắn, tương tác thừa. Trực giác cốt lõi là biểu diễn sở thích người dùng qua hai góc nhìn:

- **Sở thích ổn định:** phản ánh qua thuộc tính item và mô tả những đặc điểm tương đối bền của mỗi quan tâm.
- **Sở thích động:** phản ánh qua quan hệ đồng xuất hiện và chuyển tiếp giữa các item trong phiên.

Định nghĩa đầu vào của bài báo không phải mô hình đa hành động. Phiên được định nghĩa là chuỗi item, không bao gồm cặp (item, action_type) hay các sự kiện tìm kiếm/lọc không gắn với item. Ngoài ra, mô hình giả định mỗi item có một giá trị cho mỗi loại thuộc tính, $A_{v_i} = \{b_{i1}, \dots, b_{i|C|}\}$, trong đó C là tập loại thuộc tính. Chẳng hạn, Tmall có category, brand và seller. Ngược lại, trường properties của Trivago có thể chứa nhiều nhãn thuộc tính cho cùng một khách sạn; vì vậy, không thể đưa thẳng trường này vào phép hợp nhất thuộc tính của phương trình (9) trong bài báo.

2 Nền tảng toán học cần hiểu

Không cần đi sâu vào chứng minh để đánh giá MLEUP, nhưng cần hiểu bốn ý sau vì chúng quyết định dữ liệu nào mô hình sử dụng được.

2.1 Vector biểu diễn item và thuộc tính

Mỗi item và mỗi giá trị thuộc tính được ánh xạ thành một vector số có thể học. Hai vector có hướng gần nhau thường được mô hình xem là có quan hệ mạnh hơn. Với khách sạn, ta có thể hình dung các chiều của vector mã hóa những tổ hợp đặc điểm như loại nơi ở, tiện nghi hay mức quan tâm, dù từng chiều không có tên và không nên diễn giải riêng lẻ. MLEUP học vector item bằng quan hệ trong phiên và học vector thuộc tính từ những item mang thuộc tính đó. Do vậy, bảng ánh xạ item–thuộc tính phải đáng tin cậy; gán sai thuộc tính có thể đưa nhiễu vào cả hai nhánh.

2.2 Attention và phép tổng hợp có trọng số

Một phiên có thể chứa nhiều item nhưng không phải item nào cũng nói lên ý định hiện tại như nhau. Attention gán trọng số lớn hơn cho item hoặc thuộc tính có ích đối với ngữ cảnh, rồi lấy tổng có trọng số của các vector. Ví dụ, sau nhiều thao tác tìm kiếm, cú click gần thời điểm dự đoán có thể cung cấp tín hiệu mạnh hơn một tương tác cũ; trọng số thực tế do dữ liệu huấn luyện quyết định, không được đặt thủ công theo ví dụ này. MLEUP dùng attention ở nhánh thuộc tính, ở các nhánh đồ thị và khi trích xuất sở thích phiên.

2.3 Graph, hypergraph và thứ tự

Graph thông thường nối từng cặp item; cạnh có hướng có thể mô tả item nào xuất hiện trước item nào. Hypergraph cho phép một cạnh nối đồng thời nhiều item, phù hợp với quan hệ cùng xuất hiện trong một đoạn phiên. WHGAT học ngữ cảnh đồng xuất hiện, còn RPGAT học quan hệ trước/sau ở nhiều bậc và vị trí tương đối. Hai nhánh bổ sung hai loại thông tin khác nhau: cùng được xem trong một phiên không có nghĩa là có cùng thứ tự chuyển tiếp. Đổi lại, phải xây và lưu cấu trúc quan hệ từ dữ liệu lịch sử, nên chi phí cao hơn mô hình chỉ dùng bảng tương tác.

2.4 Điểm dự đoán và hàm mất mát

Mô hình so vector biểu diễn phiên với vector của từng item bằng tích vô hướng. Điểm càng lớn thì item càng được xếp cao; softmax biến các điểm thành phân phối dùng để huấn luyện. Loss đề xuất kéo điểm của item đúng lên, trong khi loss tự giám sát dùng các phiên gần nhau và không gần nhau để cải thiện biểu diễn thuộc tính. Hệ số λ cân bằng hai loss. Các công thức cụ thể được nêu ở Mục 4.5 của bài báo và ở phần kiến trúc dưới đây; đối với Trivago, thay đổi quan trọng nhất là tính điểm *trong danh sách ứng viên của từng clickout*, không phải trên toàn bộ kho item.

3 Kiến trúc mô hình

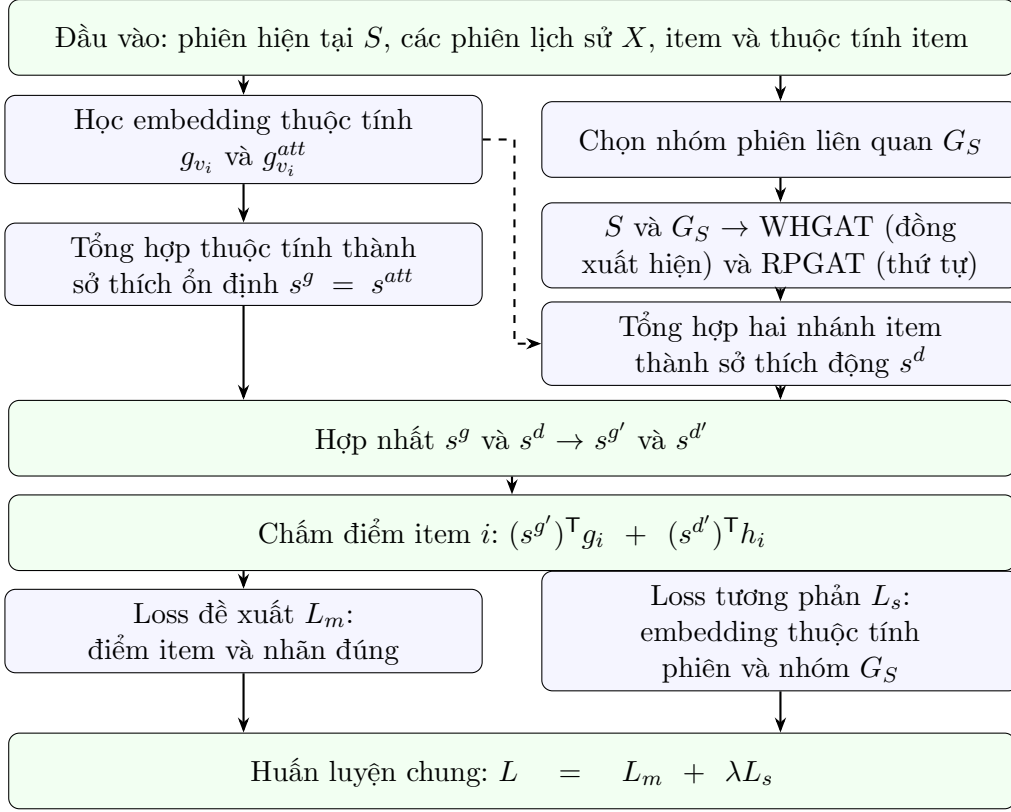
Theo Hình 1 và Mục 4 của bài báo, MLEUP có năm thành phần: (1) chọn nhóm phiên, (2) học embedding thuộc tính, (3) học embedding item qua hai nhánh quan hệ, (4) trích xuất và hợp nhất sở thích, (5) huấn luyện chung tác vụ đề xuất và tác vụ tự giám sát. Đây không phải một chuỗi năm bước nối tiếp: nhánh thuộc tính nhận dữ liệu phiên và thuộc tính item; nhóm phiên trực tiếp cấp thông tin cho hai nhánh quan hệ item và tác vụ tự giám sát.

3.1 Xây dựng nhóm phiên

Với phiên hiện tại S , mô hình ước lượng độ tương tự giữa S và các phiên lịch sử S_x bằng hiệu giữa mức chồng lặp item và mức không tương thích:

$$\text{Sim}(S, S_x) = \text{Ov}(S, S_x) - \text{In}(S, S_x),$$

rồi chọn m phiên giống nhất làm nhóm liên quan. Bài báo bỏ qua các item đuôi dài khi ước lượng quan hệ không tương thích để giảm nhiễu. Đây không đồng nghĩa với việc xóa toàn bộ item đuôi dài khỏi dữ liệu huấn luyện.



Hình 1: Luồng biểu diễn và huấn luyện của MLEUP, dựng lại từ Hình 1 và phương trình (26)–(47) của bài báo. Mũi tên nét đứt: embedding thuộc tính hướng dẫn attention khi tổng hợp hai nhánh item. L_s được tính từ embedding thuộc tính của phiên hiện tại, phiên trong G_S và phiên ngoài nhóm; nó không nhận điểm dự đoán.

3.2 Học embedding thuộc tính

Nhánh này học từ quan hệ item–thuộc tính trong dữ liệu và thuộc tính của các item trong phiên; nó không nhận G_S làm đầu vào trực tiếp. Với mỗi giá trị thuộc tính, mô hình lấy mẫu các item mang giá trị đó, tổng hợp biểu diễn bằng attention và highway gate (phương trình 4–9). Sau đó, embedding vị trí và multi-head self-attention được dùng để mô hình hóa tương tác giữa thuộc tính của các item trong cùng phiên (phương trình 10–12).

3.3 Học embedding item

Nhánh đồng xuất hiện tạo weighted session hypergraph từ các cửa sổ trượt có nhiều kích thước và các phiên trong nhóm. Thông tin được truyền theo luồng node $\rightarrow$ hyperedge $\rightarrow$ node bằng WHGAT. Nhánh thứ tự xây dựng weighted heterogeneous graph với các láng giềng trước/sau ở nhiều bậc, đưa vị trí tương đối vào RPGAT. Trọng số cạnh và hyperedge có thành phần thời gian.

3.4 Trích xuất sở thích và hợp nhất

Attention trên embedding thuộc tính của phiên tạo sở thích ổn định $s^g = s^{att}$ (phương trình 26–27). WHGAT và RPGAT tạo embedding quan hệ item; khi tổng hợp chúng thành s^{co} và s^{seq} , embedding thuộc tính item và phiên hướng dẫn attention (phương trình

28–33). Highway gate kết hợp s^{co} với s^{seq} thành sở thích động s^d (phương trình 34). Sau đó, co-guided fusion cho s^g và s^d tương tác để tạo $s^{g'}$ và $s^{d'}$ (phương trình 35–42).

3.5 Chấm điểm và huấn luyện

Điểm của item i được tính theo:

$$\text{score}_i = (s^{g'})^\top g_i + (s^{d'})^\top h_i,$$

trong đó g_i là embedding thuộc tính của item và h_i là *embedding item ban đầu*, không phải đầu ra WHGAT hoặc RPGAT (phương trình 44). Từ các điểm trên toàn tập item I , softmax và nhãn item tiếp theo tạo loss đề xuất L_m (phương trình 45–46). Song song, embedding thuộc tính của phiên hiện tại ghép với phiên trong G_S thành cặp dương, với phiên ngoài nhóm thành cặp âm để tạo loss tự giám sát L_s (phương trình 43). Hai đường huấn luyện chỉ gặp nhau tại mục tiêu $L = L_m + \lambda L_s$ (phương trình 47); L_s không được tính từ điểm dự đoán. Khi suy luận, mô hình dùng nhánh chấm điểm mà không tính hai loss. Đầu ra gốc chưa giới hạn vào danh sách ứng viên **impressions** của từng truy vấn Trivago.

4 Ưu điểm, giới hạn và bằng chứng thực nghiệm

4.1 Đánh giá định tính

Bảng 1: Đánh giá định tính các khía cạnh của MLEUP khi chuyển sang Trivago.

Khía cạnh	Đánh giá
Hiểu phiên	Phiên là đơn vị đầu vào tường minh, có thứ tự item và thông tin từ các phiên liên quan.
Thuộc tính item	Mô hình học thuộc tính và dùng chúng hướng dẫn nhánh quan hệ item. Tuy nhiên, phép kết hợp gốc giả định một giá trị cho mỗi loại thuộc tính; thuộc tính nhiều nhãn của Trivago cần bộ gộp khác.
Tương tác thưa	Thuộc tính và phiên liên quan có thể bổ sung tín hiệu. Đổi lại, tạo nhóm phiên, đồ thị và lấy mẫu item theo thuộc tính làm tăng chi phí tiền xử lý, bộ nhớ và huấn luyện. Tác giả cũng ghi nhận nhánh thuộc tính làm thời gian huấn luyện tăng.
Loại hành động	Hạn chế quan trọng khi chuyển miền: biểu diễn gốc không phân biệt interaction item image , clickout item , tìm kiếm và lọc.
Tập ứng viên	Điểm item và softmax được xác định trên tập I , không dựa trên impressions hay prices của từng clickout .

Khía cạnh	Đánh giá
Tái lập	Repo có <code>main.py</code> , <code>model.py</code> , <code>util.py</code> và thư mục cho ba bộ dữ liệu. README nêu Python 3.7, PyTorch 1.13.1 và CUDA 11.7. Việc chạy repo nguyên bản và tái lập các bảng chưa được xác minh.

4.2 Điểm mạnh theo từng hoàn cảnh

- **Phiên ngắn hoặc tương tác thưa:** thuộc tính item và nhóm phiên liên quan bổ sung ngữ cảnh mà chỉ chuỗi item hiện tại có thể không cung cấp. Lợi ích này phụ thuộc vào chất lượng thuộc tính và khả năng chọn được phiên thật sự liên quan; không phải mọi phiên ngắn đều được cải thiện.
- **Ý định đổi trong một phiên:** nhánh chuyển tiếp và vị trí tương đối theo dõi thứ tự các item; nhánh đồng xuất hiện giữ thông tin về các item được cân nhắc cùng nhau. Điều này hữu ích khi item xem gần đây phản ánh nhu cầu hiện tại.
- **Item có thuộc tính mô tả tốt:** nhánh thuộc tính có thể chia sẻ thông tin giữa các item mang đặc điểm giống nhau. Khi item ít được tương tác nhưng có metadata đáng tin cậy, đây là một nguồn tín hiệu có lý do rõ ràng để thử nghiệm.

4.3 Điểm yếu và nguyên nhân

- **Phụ thuộc vào giả định dữ liệu:** một giá trị cho mỗi loại thuộc tính là giả định cụ thể của kiến trúc gốc. Metadata nhiều nhãn hoặc lẫn kiểu giá trị không thể được nạp nguyên trạng; bộ gộp thuộc tính mới có thể làm kết quả khác bài báo.
- **Khó áp dụng cho nhật ký nhiều hành động:** một chuỗi item không phân biệt click, xem ảnh, tìm kiếm hay lọc. Nếu ép mọi sự kiện thành item, mô hình có thể hiểu sai ý định; nếu bỏ chúng, có thể mất tín hiệu quan trọng.
- **Chi phí và nguy cơ rò rỉ:** nhóm phiên và đồ thị dùng dữ liệu liên phiên, vì vậy cần tiền xử lý, bộ nhớ và quy tắc thời gian chặt chẽ. Xây đồ thị từ dữ liệu tương lai sẽ làm đánh giá lạc quan giả tạo.
- **Lợi ích không đồng đều theo metric:** kết quả và ablation trên 30music cho thấy tăng khả năng tìm đúng trong top- K không đảm bảo MRR tăng. Mô hình có nhiều nhánh nên cần đo đóng góp từng nhánh, không mặc định tất cả đều hữu ích.

Hoàn cảnh nên thử: có phiên theo thứ tự rõ ràng, nhiều phiên lịch sử có thể chia sẻ thông tin, metadata item đủ tốt và có tài nguyên để huấn luyện đồ thị. **Hoàn cảnh khó áp dụng:** thiếu chuỗi item hoặc thuộc tính, phần lớn sự kiện không gắn item, nhãn mục tiêu không phải item tiếp theo, hoặc cần xếp hạng trong danh sách ứng viên biến đổi theo từng truy vấn nhưng chưa sửa đầu ra của mô hình.

4.4 Kết quả trên các bộ dữ liệu gốc

Bài báo tạo các cặp prefix-target từ mỗi phiên, lọc item có tần suất dưới năm lần và loại phiên chỉ có một item. Các metric báo cáo gồm $P@10/20$, $M@10/20$ (MRR cắt tại K) và $N@10/20$ trên Tmall, Diginetica, 30music. Theo định nghĩa ở phương trình (48), $P@K$ là tỷ lệ truy vấn có item đúng trong top- K (thường gọi là Hit Rate@ K), không phải precision chia cho K .

Dataset	MLEUP P@20	MLEUP M@20	Diễn giải trong bài báo
Tmall	49.65	20.96	Cả P@20 và M@20 đứng đầu trong các mô hình được so sánh.
Diginetica	55.39	19.43	Cả hai metric đứng đầu; chênh lệch M@20 với GCE-GNN (19.04) tương đối nhỏ.
30music	46.18	19.44	P@20 đứng đầu, nhưng M@20 thấp hơn GCE-GNN (23.42) và một số baseline khác.

Bảng 2: Một số kết quả MLEUP ở Bảng 3 của bài báo; đơn vị phần trăm. Đây không phải kết quả trên Trivago.

Không thể khái quát rằng MLEUP luôn tốt nhất về MRR. Trong ablation trên 30music, bỏ session group cho M@20 = 21.26 so với 19.44 của mô hình đầy đủ (Bảng 5); bỏ nhánh thuộc tính hoặc loss tự giám sát cũng làm M@20 tăng (Bảng 6). Bảng 4 còn cho thấy bỏ nhánh đồng xuất hiện có thể nâng MRR ở Tmall và 30music. Đây là bằng chứng cần chú ý khi mục tiêu là MRR, dù bỏ sung thuộc tính có thể cải thiện Precision. Tất cả các số này thuộc các bộ dữ liệu gốc trong bài báo.

5 Đối chiếu với dữ liệu Trivago

Trong `train.csv`, mỗi hàng là một sự kiện có các trường như `session_id`, `step`, `action_type`, `reference`, và đôi khi có `impressions/prices`. Một số giá trị `reference` là địa điểm hoặc từ khóa thay vì mã khách sạn. Trong `item_metadata.csv`, một `item_id` có thể có nhiều `properties` phân cách bởi dấu `|`, chẳng hạn WiFi, Pool, rating hoặc loại khách sạn. Do đó, cần phân biệt sự kiện gắn với khách sạn với sự kiện tìm kiếm/lọc, đồng thời xử lý thuộc tính nhiều nhân.

Ví dụ từ dữ liệu hiện có. Hàng 15 của `train.csv` là một `clickout item` trong phiên `aff3928535f48`; `reference=109038` nằm ở vị trí 10 của 25 `impressions`, và `prices` ở đúng vị trí đó là 56. Ví dụ này cho thấy nhân là khách sạn được click trong danh sách đã hiển thị; giá phải ghép theo vị trí. Nó không chứng minh người dùng đã đặt phòng. Một `session_id` có thể chứa nhiều `clickout`, nên mỗi lượt `clickout` cần một `query_id` riêng, gắn với tệp nguồn và vị trí hàng để tránh nhầm truy vấn.

Bảng 3: Đối chiếu giả định MLEUP với yêu cầu dữ liệu Trivago.

Yêu cầu Trivago	MLEUP gốc	Điều chỉnh cần thiết
Lịch sử trong phiên	Chuỗi item có thứ tự.	Dựng prefix sự kiện trước từng <code>clickout</code> ; giữ thứ tự <code>step</code> và xử lý nhiều <code>clickout</code> trong một phiên.
Nhiều loại hành động	Không có trong định nghĩa đầu vào gốc.	Thêm embedding/cơ chế truyền thông tin <code>action_type</code> ; biểu diễn riêng sự kiện tìm kiếm và lọc không gắn item.

Yêu cầu Trivago	MLEUP gốc	Điều chỉnh cần thiết
Thuộc tính nhiều giá trị	Phương trình (9) ghép một giá trị mỗi loại thuộc tính.	Tách nhãn, chuẩn hóa từ vựng; thay phép gộp bằng masked pooling, attention hoặc biểu diễn quan hệ thừa. Nếu gom thuộc tính theo loại, vẫn phải hỗ trợ nhiều giá trị trong một loại.
Clickout và ứng viên	Chấm điểm trên toàn tập item I .	Chỉ xếp hạng ứng viên được hiển thị; giữ ghép <code>impressions-prices</code> theo vị trí; huấn luyện theo truy vấn. Giá có thể là đặc trưng động của ứng viên.
Đánh giá	M@10/20 trên prefix-target của ba bộ dữ liệu riêng.	Chia theo thời gian, chỉ dùng lịch sử trước truy vấn, tính MRR theo truy vấn clickout và so với baseline item-only.

Ví dụ, mỗi hàng `clickout item` tạo một truy vấn; `reference` là khách sạn được click, còn `impressions` là các ứng viên đã hiển thị. Các sự kiện đứng trước hàng đó trong cùng phiên tạo ngữ cảnh. Metadata bổ sung thuộc tính cho khách sạn trong lịch sử và cho các ứng viên. Vì một `session_id` có thể chứa nhiều clickout, nên định danh truy vấn cần gắn với `source_file` và chỉ số hàng hoặc `query_id`. Đây là thiết kế đề xuất cho Trivago, không phải pipeline sẵn có trong bài báo.

6 Điều kiện sử dụng và yêu cầu tài nguyên

6.1 Điều kiện dữ liệu

Để huấn luyện một biến thể MLEUP cho Trivago, trước tiên phải tạo tập truy vấn với bốn thành phần đồng nhất: (1) lịch sử sự kiện trước thời điểm clickout; (2) danh sách khách sạn đã hiển thị; (3) khách sạn được click làm nhãn; (4) metadata của các khách sạn trong lịch sử và danh sách ứng viên. Khi tách `impressions` và `prices`, phải giữ cả các token rỗng trước khi kiểm tra vị trí để không làm lệch cặp item-giá. Sự kiện tìm kiếm/lọc không gắn mã khách sạn cần một cách biểu diễn riêng; không thể nối trực tiếp mọi `reference` với `item_metadata.csv`.

Việc chọn nhóm phiên, xây đồ thị và học từ metadata phải tuân theo mốc thời gian của tập huấn luyện/đánh giá. Nếu một phiên hoặc một cạnh đồ thị chứa tương tác sau clickout đang dự đoán, mô hình đã nhìn thấy tương lai. Với phiên rất ngắn, item ít phổ biến hoặc metadata thiếu, cần quy tắc dự phòng và báo cáo kết quả riêng theo từng nhóm. Tập `submission_popular.csv` là ví dụ đầu ra, không phải nhãn để huấn luyện.

6.2 Môi trường phần mềm và cấu hình máy

README của repo tác giả nêu Python 3.7, PyTorch 1.13.1 và CUDA 11.7 cho môi trường gốc. Đây là thông tin để thử tái lập, *không phải* cấu hình tối thiểu đã được đo cho Trivago. Các tệp Trivago hiện có tại workspace gồm `train.csv` khoảng 2.10 GB,

`test.csv` khoảng 0.53 GB và `item_metadata.csv` khoảng 0.26 GB trên đĩa. Bảng sự kiện sau giải mã, embedding, nhóm phiên và đồ thị có thể chiếm bộ nhớ lớn hơn nhiều kích thước CSV; không thể suy VRAM/RAM cần thiết chỉ từ dung lượng tệp.

Quy trình khả thi là đọc CSV theo khối để dựng truy vấn và từ điển item/thuộc tính, thử mô hình trên tập con theo thời gian, đo RAM/VRAM và thời gian cho từng bước, rồi mới tăng quy mô. GPU có thể cần cho huấn luyện đồ thị quy mô lớn, nhưng chưa có căn cứ để ấn định một mức VRAM tối thiểu hay thời gian chạy. Nếu chỉ dùng CPU, nên bắt đầu với tập con và các baseline đơn giản để kiểm tra logic dữ liệu trước.

6.3 Đánh giá mức phù hợp

Phù hợp về tín hiệu đầu vào: Trivago có lịch sử theo phiên, thứ tự `step`, item được click và metadata khách sạn; đây là những nguồn thông tin MLEUP muốn khai thác. **Chưa khớp về cấu trúc:** nhánh thuộc tính cần hỗ trợ nhiều nhãn, chuỗi phiên cần mã hóa nhiều loại hành động, còn hàm chấm điểm phải bị ràng buộc bởi `impressions`. **Chưa biết hiệu quả thực tế:** MRR Trivago và chi phí chạy chưa được đo. Vì vậy, mức phù hợp tổng thể là *có tiềm năng nghiên cứu nhưng cần sửa kiến trúc nhiều*; không thể gọi là áp dụng trực tiếp hoặc kết luận hiệu quả cao trước thí nghiệm.

7 Tính khả thi và lộ trình kiểm chứng

7.1 Nhận định kỹ thuật

MLEUP là nền tảng nghiên cứu hợp lý khi trọng tâm là ý định trong phiên kết hợp thuộc tính item. Mức phù hợp trực tiếp với Trivago thấp hơn vì ba điểm: công thức thuộc tính giả định một giá trị mỗi loại; biểu diễn gốc thiếu loại hành động; đầu ra không xếp hạng trong danh sách `impressions`. Kết quả 30music cũng cho thấy thêm thông tin có thể làm Precision tăng nhưng MRR giảm. Vì vậy, hiện chưa có căn cứ để kết luận MLEUP sẽ cải thiện MRR Trivago hoặc vượt baseline trước khi có thí nghiệm đối chứng.

Về kỹ thuật, có thể xây dựng biến thể nhưng mức thay đổi kiến trúc đáng kể nếu muốn giữ đầy đủ MLEUP. Cần sửa bộ mã hóa thuộc tính, biểu diễn sự kiện không gắn mã khách sạn, và đầu ra/loss theo danh sách ứng viên. Nhóm phiên, đồ thị và metadata phải được dựng chỉ từ dữ liệu huấn luyện và lịch sử hợp lệ tại thời điểm dự đoán để tránh rò rỉ tương lai. Chưa thể ước lượng đáng tin cậy thời gian chạy hoặc nhu cầu GPU nếu chưa chạy mã và đo kích thước đồ thị Trivago.

7.2 Các bước kiểm chứng đề xuất

1. Chạy mã gốc trên một dataset đi kèm repo; ghi lại môi trường, seed và metric. Việc này chưa được thực hiện.
2. Dựng baseline Trivago chỉ từ tương tác item; tạo một truy vấn cho mỗi clickout và chỉ xếp hạng `impressions`. So MRR với baseline đơn giản, phổ biến.
3. Thay phép gộp thuộc tính bằng bộ mã hóa `properties` nhiều nhãn; đo riêng tác động lên MRR và Precision.
4. Thêm `action_type`, rồi mở rộng sang tìm kiếm/lọc không gắn item; làm ablation `item-only`, `+properties`, `+actions`, `+both`.
5. Kiểm tra rò rỉ thời gian ở nhóm phiên, đồ thị và metadata; phân tích MRR theo độ dài phiên, độ phổ biến item và mức thiếu thuộc tính.

8 Ranh giới xác minh và kết luận

Các phần đã được đối chiếu với PDF của bài báo gồm năm thành phần trong Hình 1; định nghĩa phiên và thuộc tính; WHGAT/RPGAT; trích xuất và hợp nhất sở thích; điểm item và loss; ba dataset, $K = 10/20$, Bảng 3–6; cùng giới hạn thời gian huấn luyện mà tác giả nêu. Bài báo báo cáo trên Tmall, Diginetica và 30music, không thử nghiệm Trivago.

Chưa xác minh bằng thực thi việc repo chạy được trên môi trường hiện tại, khả năng khớp Bảng 3, chi phí trên Trivago hay lợi ích của các thay đổi về thuộc tính nhiều giá trị, loại hành động và ranking trong `impressions`. Các kết luận đó cần thực nghiệm.

Kết luận. Có thể triển khai một biến thể từ MLEUP, nhưng đây là dự án sửa kiến trúc đáng kể. Chỉ nên chọn làm mô hình chính sau khi tái lập được mã gốc và đo MRR theo truy vấn clickout trên Trivago. Ablation là cần thiết vì một số nhánh bổ sung từng làm giảm MRR trên 30music.

Tài liệu tham khảo

- [1] L. Zhang, D. Shen, Y. Kou, T. Nie, “Multi-perspective learning for enhanced user preferences for session-based recommendation,” *Knowledge-Based Systems*, vol. 298, article 111997, 2024. DOI: [10.1016/j.knosys.2024.111997](https://doi.org/10.1016/j.knosys.2024.111997).
- [2] Mã nguồn MLEUP của tác giả: <https://github.com/lean030/MLEUP>.
- [3] Trivago RecSys Challenge 2019, dữ liệu huấn luyện và metadata tại workspace: `Data/Tourism/Trivago_RecSys_Challenge_2019/raw/`.